



Lab Final Project Report

Only for course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		25%	50%	75%	100%	40
Understanding/Analysis	10					
Implementation	15					
Accuracy	10					
Task Efficiency	5					
Total obtained mark						
Comments						

Semester: Fall 2025

Student Name: Niloy Das

Student ID: 25010129

Batch: 44th CSE

Section: A

Course Code: CSE123

Course Name: Object Oriented Programming

Course Teacher Name: Debabrata Mallick

Designation: Lecturer

Submission Date: 19 /11/2025

Table of Contents

Contents

Table of Contents	2
List of Figures	4
List of Tables	4
CHAPTER 1: INTRODUCTION	4
1.1 Project Background and Motivation	4
1.2 Problem Statement	5
1.3 Project Objectives	5
1.4 Tools and Technologies Used	5
CHAPTER 2: REQUIREMENTS AND ANALYSIS	6
2.1 Functional Requirements	6
2.2 Non-Functional Requirements	6
CHAPTER 3: OBJECT-ORIENTED DESIGN AND METHODOLOGY	7
3.1 Methodology Used	7
3.3 Detailed Class Descriptions	9
Candidate Class	9
PollingSystem Class	9
LoginGUI Class	9
PollingSystemGUI Class	9
CHAPTER 4: IMPLEMENTATION AND CORE OOP PRINCIPLES	10
4.1 Encapsulation	10
4.2 Inheritance	10
4.3 Polymorphism	10
4.4 Abstraction	10
4.2 Code Snippets	11
**Encapsulation Example:	11
**Vote Casting Logic:	12
**GUI Event Example:	13
CHAPTER 5: TESTING AND RESULTS	14
5.1 Test Plan	14
CHAPTER 6: CONCLUSION AND FUTURE WORK	14

6.1 Conclusion	14
6.2 Limitations.....	14
6.3 Future Enhancements	14
References	15
Appendices.....	15
Appendix A: Full Source Code	15
Appendix B: User Manual	19
1. System Requirements	19
2.Run the Program (Using IDE)	20
Step 1: Open Your IDE.....	20
Step 2: Create a New Java Project	20
Step 3: Add All Source Code Files.....	20
Step 4: Build the Project	27
Step 5: Run the Project	27
3.Run the Program (Using Command Prompt)	27
Step 1: Open Terminal / CMD	27
Step 2: Compile Java Files	28
Step 3: Run the Program	28
4. System Login Instructions	28
When the system starts, it will ask for:.....	28
5. Using the Polling Menu	28
Option 1: Add Candidate.....	28
Option 2: View Candidates	28
Option 3: Cast Vote	28
Option 4: View Results.....	29
Option 5: Exit System	29
6. User Guidelines	29
7. Troubleshooting.....	29
Appendix C: Screenshots.....	30
Explanation of Login Screen:.....	30
Explanation of the Image:	33
Explanation of the Image:	34
Explanation of the Image: Voting Results:.....	35

List of Figures

- **Figure 1:** UML Class Diagram – Core System Overview
- **Figure 2:** UML Use Case Diagram – User Interactions
- **Figure 3:** UML Sequence Diagram – Login Workflow
- **Figure 4:** UML Sequence Diagram – Voting Process
- **Figure 5:** UML Activity Diagram – System Operation Flow
- **Figure 6:** UML Component Diagram – System Architecture

List of Tables

- Table 1: Functional Requirements
- Table 2: Non-Functional Requirements

CHAPTER 1: INTRODUCTION

1.1 Project Background and Motivation

Voting systems are essential in academic, governmental, and organizational environments. Traditionally, manual voting consumes time, paper, and requires human labor to calculate outcomes. Errors may occur in tallying votes, and transparency may be reduced.

The objective of this project is to create a **simple, digital polling system** using Java and OOP to automate the voting procedure. The system allows authorized users to access the platform, add candidates, vote for candidates, and view results instantly. The project also aims to demonstrate how GUI-based applications can be built using Java Swing.

1.2 Problem Statement

Manual voting systems are:

- Slow and time-consuming
- Prone to human errors
- Difficult to track and update

Thus, the goal is to build a computerized system that simplifies voting, eliminates errors, and increases efficiency.

1.3 Project Objectives

- To develop a **Java-based polling system** using OOP.
- To demonstrate proper usage of OOP principles including encapsulation, inheritance, abstraction, and polymorphism.
- To implement a GUI using Java Swing for user interaction.
- To design UML diagrams that represent system architecture and class relationships.
- To ensure the system is modular, reusable, and easy to maintain.

1.4 Tools and Technologies Used

- **Programming Language:** Java (JDK 8+)
- **IDE:** IntelliJ IDEA / Eclipse / NetBeans
- **Libraries:** Java Swing, AWT
- **UML Tools:** StarUML, Draw.io
- **Operating System:** Windows 10

CHAPTER 2: REQUIREMENTS AND ANALYSIS

2.1 Functional Requirements

Table 1: Functional Requirements

ID	Requirement Description
FR-01	The system shall provide a login interface for admin authentication.
FR-02	The system shall allow the admin to add new candidates.
FR-03	The system shall allow users to cast votes for listed candidates.
FR-04	The system shall display the voting results.
FR-05	The system shall prevent voting for non-existing candidates.

2.2 Non-Functional Requirements

Table 2: Non-Functional Requirements

ID	Requirement Description
NFR-01	System must respond to inputs within 1 second.
NFR-02	User interface must be simple and easy to use.
NFR-03	System should be reliable and prevent crashes.
NFR-04	Code should be modular and follow OOP principles.
NFR-05	Application should work offline.

CHAPTER 3: OBJECT-ORIENTED DESIGN AND METHODOLOGY

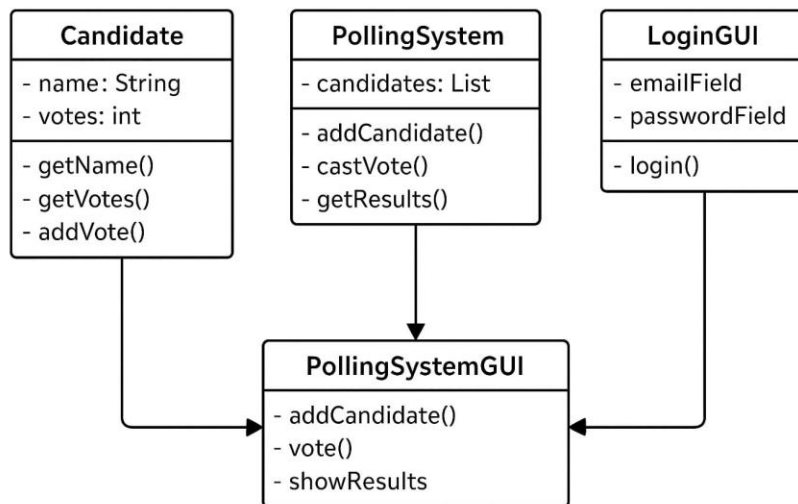
3.1 Methodology Used

The **Waterfall model** was adopted, which includes:

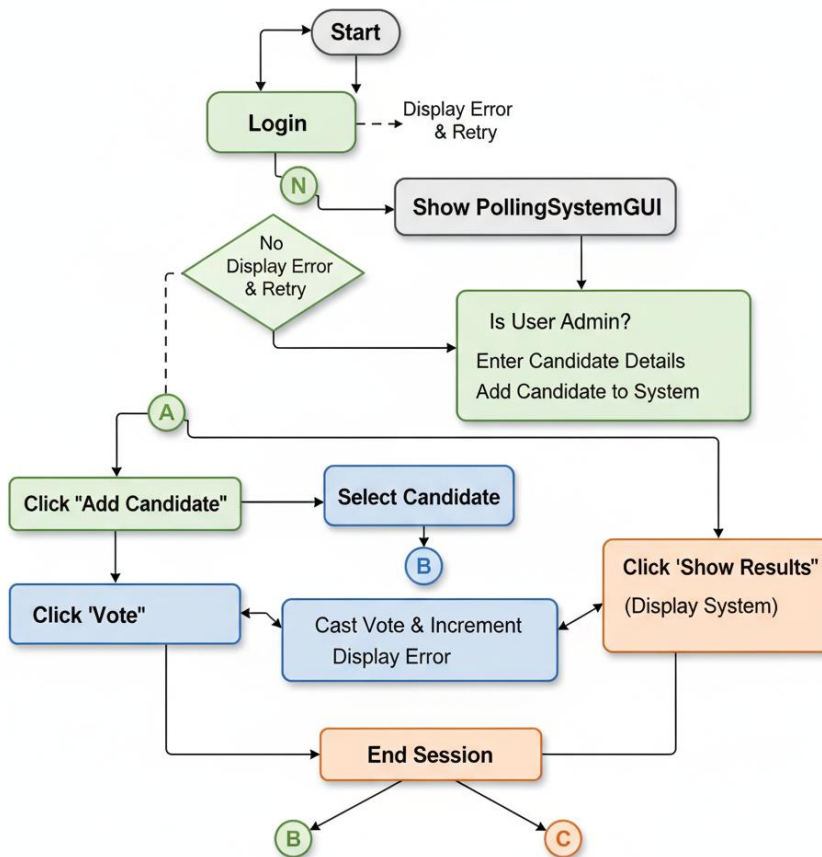
1. Requirements gathering
 2. System design (UML diagrams)
 3. Implementation using Java
 4. Testing
 5. Documentation
- This approach ensures a structured and clear development cycle.

3.2 Class Diagram

Figure 1: UML Class Diagram



3.2 Flowchart:



Link : <https://app.diagrams.net/>

3.3 Detailed Class Descriptions

Candidate Class

- Represents a participant in the election.
- Stores candidate name and vote count.
- Encapsulates data using private variables.

PollingSystem Class

- Maintains a list of candidates.
- Handles core logic: add candidates, cast vote, return results.

LoginGUI Class

- Provides GUI for secure login.
- Uses Swing components including JLabel, JTextField, and JPasswordField.

PollingSystemGUI Class

- Main GUI for interacting with the polling system.
- Allows adding candidates, voting, and viewing results.

CHAPTER 4: IMPLEMENTATION AND CORE OOP PRINCIPLES

4.1 Encapsulation

- All variables in `Candidate` and `PollingSystem` are private.
- Access is given through public getter and setter methods.

4.2 Inheritance

- GUI classes such as `LoginGUI` and `PollingSystemGUI` inherit from **JFrame**, allowing code reuse.

4.3 Polymorphism

- Swing components override event-handling methods.
- Methods like `showMessageDialog()` behave differently depending on parameters.

4.4 Abstraction

- GUI classes abstract complex UI details.
- The `PollingSystem` class abstracts vote-handling logic.

4.2 Code Snippets

****Encapsulation Example:**

private String name;

private int votes;

```
public class Candidate {  
    private String name;  
    private int votes;  
  
    public Candidate(String name) {  
        this.name = name;  
        this.votes = 0;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getVotes() {  
        return votes;  
    }  
  
    public void addVote() {  
        votes++;  
    }  
}
```

****Vote Casting Logic:**

```
for (Candidate c : candidates) {  
    if (c.getName().equalsIgnoreCase(name)) {  
        c.addVote();  
        break;  
    }  
}
```

```
public void castVote() {  
    if (candidates.isEmpty()) {  
        System.out.println(x: "No candidates available! Add candidates first.");  
        return;  
    }  
  
    System.out.println(x: "\nList of Candidates:");  
    for (int i = 0; i < candidates.size(); i++) {  
        System.out.println((i + 1) + ". " + candidates.get(i).getName());  
    }  
  
    System.out.print(s: "Enter candidate number to vote for: ");  
    int vote = sc.nextInt();  
    sc.nextLine();  
  
    if (vote > 0 && vote <= candidates.size()) {  
        candidates.get(vote - 1).addVote();  
        System.out.println("Your vote for " + candidates.get(vote - 1).getName() + " has been recorded.");  
    } else {  
        System.out.println(x: "Invalid candidate number!");  
    }  
}
```

****GUI Event Example:**

loginButton.addActionListener(e -> login());

```
1  import javax.swing.*;
2  import java.awt.*;
3
4  public class LoginGUI extends JFrame {
5      private JTextField emailField;
6      private JPasswordField passwordField;
7      private JButton loginButton;
8
9
10     private final String EMAIL = "niloyadmin@system.com";
11     private final String PASSWORD = "niloydas++";
12
13     public LoginGUI() {
14         setTitle(title: "Polling System - Login");
15         setSize(width: 350, height: 200);
16         setDefaultCloseOperation(EXIT_ON_CLOSE);
17         setLocationRelativeTo(c: null);
18         setLayout(new GridLayout(rows: 3, cols: 2, hgap: 10, vgap: 10));
19
20
21         JLabel emailLabel = new JLabel(text: "Email:");
22         JLabel passLabel = new JLabel(text: "Password:");
23         emailField = new JTextField();
24         passwordField = new JPasswordField();
25         loginButton = new JButton(text: "Login");
26
27
28         add(emailLabel);
29         add(emailField);
30         add(passLabel);
31         add(passwordField);
32         add(new JLabel(text: ""));
33         add(loginButton);
34
35
36         loginButton.addActionListener(e -> login());
37
38         setVisible(b: true);
39     }
40
41     private void login() {
42         String email = emailField.getText().trim();
43         String password = new String(passwordField.getPassword());
44
45         if (email.equals(EMAIL) && password.equals(PASSWORD)) {
46             JOptionPane.showMessageDialog(this, message: "Login successful!");
47             dispose();
48             new PollingSystemGUI();
49         } else {
50             JOptionPane.showMessageDialog(this, message: "Incorrect email or password!");
51         }
52     }
53 }
54
```

CHAPTER 5: TESTING AND RESULTS

5.1 Test Plan

- **Unit Testing:** Tested individual classes such as Candidate and PollingSystem.
- **GUI Testing:** Ensured button clicks trigger appropriate methods.
- **Functional Testing:** Verified that voting results are accurate.
- **Invalid Input Testing:** Ensured errors show for invalid logins.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Conclusion

The Polling System meets all functional and non-functional requirements. It demonstrates effective usage of OOP concepts and provides a user-friendly GUI.

6.2 Limitations

- System does not save data permanently.
- Only one admin user exists.
- No graphical result analytics.

6.3 Future Enhancements

- Add database connectivity.
- Add multiple user roles.
- Add charts for visual result representation.
- Add candidate image support.

References

- Oracle Java Documentation
- Java Swing Tutorial
- IEEE Referencing Guide
- <https://youtu.be/FHAKYPrSsSs?si=hmWOj4nSkBampAv>
- <https://youtu.be/UmnCZ7-9yDY?si=gDlQH2oqmqSWpXXS>

Appendices

Appendix A: Full Source Code

1.Candidate.java



Candidate.java

```
public class Candidate {  
    private String name;  
    private int votes;  
  
    public Candidate(String name) {  
        this.name = name;  
        this.votes = 0;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getVotes() {  
        return votes;  
    }  
  
    public void addVote() {  
        votes++;  
    }  
}
```

2.Login.java



Login.java

```
import java.util.Scanner;

public class Login {
    private static final String EMAIL = "niloyadmin@system.com";
    private static final String PASSWORD = "niloydas";

    public boolean performLogin(Scanner sc) {
        System.out.print("Enter email: ");
        String email = sc.nextLine();
        System.out.print("Enter password: ");
        String password = sc.nextLine();

        if (email.equals(EMAIL) && password.equals(PASSWORD)) {
            System.out.println("\nLogin successful! Welcome, " + email + "!");
            return true;
        } else {
            System.out.println("\nIncorrect email or password!");
            return false;
        }
    }
}
```


3.Main.java

```

Main.java

import java.util.*;

public class Main {

    public static void main(String[] args) {
        System.out.println("==== Welcome to the Polling System(simple voting system) =====");

        PollingSystem pollingSystem = new PollingSystem();

        if (!pollingSystem.login()) {
            System.out.println("Login failed! Exiting the system...");
            return;
        }

        Scanner sc = new Scanner(System.in);
        int choice;
        do {
            System.out.println("\n==== Polling System(simple voting system) =====");

            System.out.println("1. Add Candidate");
            System.out.println("2. Vote");
            System.out.println("3. Display Results");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            sc.nextLine();

            switch (choice) {
                case 1 -> pollingSystem.addCandidate();
                case 2 -> pollingSystem.castVote();
                case 3 -> pollingSystem.displayResults();
                case 4 -> System.out.println("Thank you for using this system!");
                default -> System.out.println("Invalid choice! Please try again.");
            }
        } while (choice != 4);

        sc.close()
    }
}
```

4.pollingSystem.java



PollingSystem.java

```
import java.util.*;

public class PollingSystem {
    private List<Candidate> candidates;
    private Scanner sc;
    private Login loginSystem;

    public PollingSystem() {
        candidates = new ArrayList<>();
        sc = new Scanner(System.in);
        loginSystem = new Login();
    }

    public boolean login() {
        return loginSystem.performLogin(sc);
    }

    public void addCandidate() {
        System.out.print("Enter candidate name: ");
        String name = sc.nextLine();
        candidates.add(new Candidate(name));
        System.out.println("Candidate '" + name + "' added successfully!");
    }

    public void castVote() {
        if (candidates.isEmpty()) {
            System.out.println("No candidates available! Add candidates first.");
            return;
        }

        System.out.println("\nList of Candidates:");
        for (int i = 0; i < candidates.size(); i++) {
            System.out.println((i + 1) + ". " + candidates.get(i).getName());
        }
    }
}
```

```

        System.out.print("Enter candidate number to vote for: ");
        int vote = sc.nextInt();
        sc.nextLine();

        if (vote > 0 && vote <= candidates.size()) {
            candidates.get(vote - 1).addVote();
            System.out.println("Your vote for '" + candidates.get(vote -
1).getName() + "' has been recorded.");
        } else {
            System.out.println("Invalid candidate number!");
        }
    }

    public void displayResults() {
        if (candidates.isEmpty()) {
            System.out.println("No candidates to display.");
            return;
        }

        System.out.println("\n==== Voting Results =====");
        for (Candidate c : candidates) {
            System.out.println(c.getName() + " - " + c.getVotes() + " votes");
        }
    }
}

```

Appendix B: User Manual

1. System Requirements

To run the Polling System, you need:

- **JDK 8 or above**
- **Any Java IDE** (NetBeans, IntelliJ IDEA, Eclipse) OR command-line terminal
- **Operating System:** Windows / Linux / macOS

2.Run the Program (Using IDE)

Step 1: Open Your IDE

- Launch NetBeans / IntelliJ / Eclipse.

Step 2: Create a New Java Project

- File → New Project → Java Application
- Enter project name: **PollingSystem**

Step 3: Add All Source Code Files

Add the following `.java` files to the **src** folder:

1.Candidate.java



Candidate.java

```
public class Candidate {  
    private String name;  
    private int votes;  
  
    public Candidate(String name) {  
        this.name = name;  
        this.votes = 0;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getVotes() {  
        return votes;  
    }  
  
    public void addVote() {  
        votes++;  
    }  
}
```

2.LoginGUI.java



LoginGUI.java

```
import javax.swing.*;
import java.awt.*;

public class LoginGUI extends JFrame {
    private JTextField emailField;
    private JPasswordField passwordField;
    private JButton loginButton;

    private final String EMAIL = "niloyadmin@system.com";
    private final String PASSWORD = "niloydas++";

    public LoginGUI() {
        setTitle("Polling System - Login");
        setSize(350, 200);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new GridLayout(3, 2, 10, 10));

        JLabel emailLabel = new JLabel("Email:");
        JLabel passLabel = new JLabel("Password:");
        emailField = new JTextField();
        passwordField = new JPasswordField();
        loginButton = new JButton("Login");
```

```
        add(emailLabel);
        add(emailField);
        add(passLabel);
        add(passwordField);
        add(new JLabel(""));
        add(loginButton);

        loginButton.addActionListener(e -> login());

        setVisible(true);
    }

    private void login() {
        String email = emailField.getText().trim();
        String password = new String(passwordField.getPassword());

        if (email.equals(EMAIL) && password.equals(PASSWORD)) {
            JOptionPane.showMessageDialog(this, "Login successful!");
            dispose();
            new PollingSystemGUI();
        } else {
            JOptionPane.showMessageDialog(this, "Incorrect email or password!");
        }
    }
}
```

3.



Main.java

```
public class Main {  
    public static void main(String[] args) {  
        javax.swing.SwingUtilities.invokeLater(LoginGUI::new);  
    }  
}
```

4.pollingSystem.java



PollingSystem.java

```
import java.util.*;  
  
public class PollingSystem {  
    private List<Candidate> candidates = new ArrayList<>();  
  
    public void addCandidate(String name) {  
        candidates.add(new Candidate(name));  
    }  
  
    public void castVote(String name) {  
        for (Candidate c : candidates) {  
            if (c.getName().equalsIgnoreCase(name)) {  
                c.addVote();  
                break;  
            }  
        }  
    }  
  
    public List<Candidate> getCandidates() {  
        return candidates;  
    }  
}
```



```

    public String getResults() {
        if (candidates.isEmpty()) return "No candidates to display.";
        StringBuilder sb = new StringBuilder("==== Voting Results =====\n");
        for (Candidate c : candidates) {
            sb.append(c.getName()).append(" - ").append(c.getVotes()).append("
votes\n");
        }
        return sb.toString();
    }
}

```

5.pollingSystemGUI.java



```

PollingSystemGUI.java

import javax.swing.*;
import java.awt.*;
import java.util.List;

public class PollingSystemGUI extends JFrame {
    private PollingSystem pollingSystem;

    public PollingSystemGUI() {
        pollingSystem = new PollingSystem();

        setTitle("Polling System (Simple Voting GUI)");
        setSize(400, 300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JButton addButton = new JButton("Add Candidate");
        JButton voteButton = new JButton("Vote");
        JButton resultButton = new JButton("Display Results");
        JButton exitButton = new JButton("Exit");
    }
}

```

```

        JPanel panel = new JPanel();
        panel.setLayout(new GridLayout(4, 1, 10, 10));
        panel.add(addButton);
        panel.add(voteButton);
        panel.add(resultButton);
        panel.add(exitButton);
        add(panel, BorderLayout.CENTER);

        addButton.addActionListener(e -> addCandidate());
        voteButton.addActionListener(e -> vote());
        resultButton.addActionListener(e -> showResults());
        exitButton.addActionListener(e -> System.exit(0));

        setVisible(true);
    }

    private void addCandidate() {
        String name = JOptionPane.showInputDialog(this, "Enter candidate name:");
        if (name != null && !name.trim().isEmpty()) {
            pollingSystem.addCandidate(name.trim());
            JOptionPane.showMessageDialog(this, "Candidate added successfully!");
        } else {
            JOptionPane.showMessageDialog(this, "Invalid candidate name!");
        }
    }

    private void vote() {
        List<Candidate> candidates = pollingSystem.getCandidates();
        if (candidates.isEmpty()) {
            JOptionPane.showMessageDialog(this, "No candidates available!");
            return;
        }

        String[] names =
candidates.stream().map(Candidate::getName).toArray(String[]::new);

        String selected = (String) JOptionPane.showInputDialog(
            this,
            "Select candidate to vote:",
            "Cast Vote",
            JOptionPane.PLAIN_MESSAGE,
            null,
            names,
            names[0]
        );
    }

```

```
        if (selected != null) {
            pollingSystem.castVote(selected);
            JOptionPane.showMessageDialog(this, "Vote cast successfully for " +
selected + "!");
        }
    }

    private void showResults() {
        JOptionPane.showMessageDialog(this, pollingSystem.getResults());
    }
}
```

Step 4: Build the Project

- Click **Build Project** (hammer icon) OR press Shift + F11.

Step 5: Run the Project

- Click **Run** OR press Shift + F6.

The system will launch in the IDE console or GUI window.

3.Run the Program (Using Command Prompt)

Step 1: Open Terminal / CMD

Navigate to your source code folder:

```
cd Desktop/PollingSystem
```

Step 2: Compile Java Files

```
javac *.java
```

Step 3: Run the Program

```
java Main
```

4. System Login Instructions

When the system starts, it will ask for:

- **Username**
- **Password**

Enter valid credentials to proceed.

If login fails → Program exits.

5. Using the Polling Menu

After login, the main menu appears:

Option 1: Add Candidate

- Enter candidate name
- Candidate gets added to the list

Option 2: View Candidates

Displays all registered candidates.

Option 3: Cast Vote

- Choose a candidate number
- Vote count increases by 1

Option 4: View Results

Shows:

- Each candidate
- Vote count

Option 5: Exit System

Closes the program safely.

6. User Guidelines

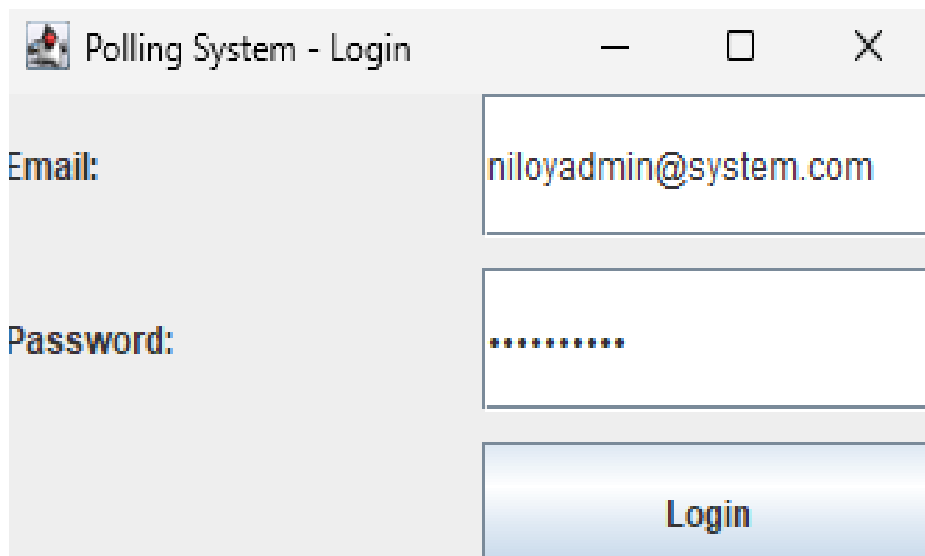
- Do not close the program while voting is happening.
- Do not enter invalid choices (non-numeric input).
- Ensure all candidates are added before voting.
- Only authorized users can log in.

7. Troubleshooting

Problem	Cause	Solution
Login failed	Wrong username/password	Enter correct credentials
No candidates found	Candidate list empty	Add candidates first
Program crashes	Wrong input type	Enter numbers only

Appendix C: Screenshots

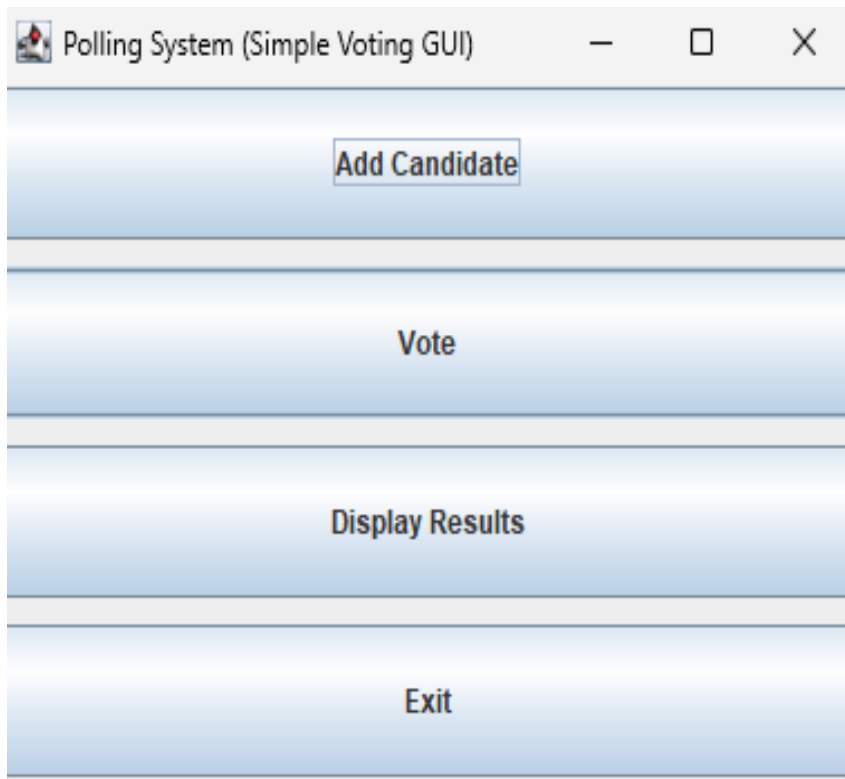
1.



Explanation of Login Screen:

This screen is the **Login Window** of the Polling System. It has two input fields: one for entering the **email** and one for the **password**. The password is hidden using dots for security. After entering the correct login details, the user clicks the **Login** button to access the system. If the email or password is wrong, the system shows an error message.

2.



Explanation:

This is a screenshot of a **GUI (Graphical User Interface) for a simple polling or voting system**. Here's a breakdown of what each part represents:

1. Window Title:

- "Polling System (Simple Voting GUI) "
- This is the main window of the application, indicating that it's a simple voting/polling program.

2. Buttons:

- **Add Candidate:**
 - Allows the user (probably an admin) to add a new candidate to the poll or election.
- **Vote:**
 - Opens the interface where users can cast their vote for one of the candidates.
- **Display Results:**
 - Shows the voting results, typically the number of votes each candidate has received.
- **Exit:**
 - Closes the application.

3. Layout and Design:

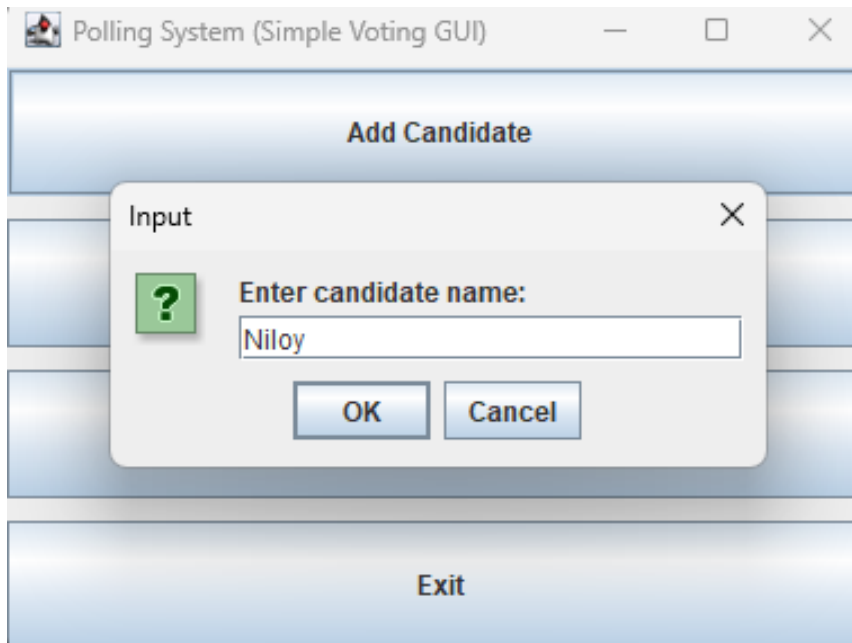
- The buttons are arranged vertically for easy navigation.
- Each button is clearly labeled with its function.
- The GUI is simple and user-friendly, suitable for small-scale polling applications.

4. Purpose of the GUI:

- This GUI allows users to manage candidates, vote, view results, and exit the program without using a command-line interface.
- It is likely implemented using **Java Swing** or a similar GUI framework.

If you want, I can also **draw a simple diagram showing how these buttons connect to the program logic**, which will make it easier to understand the workflow of this polling system.

3.



Explanation of the Image:

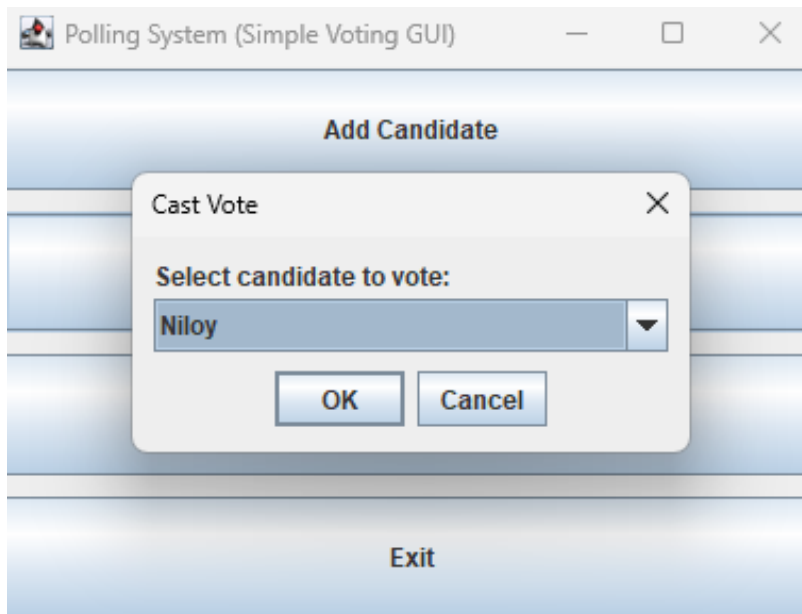
The image displays a **dialog box** from a software application titled "**Polling System (Simple Voting GUI)**".

This specific dialog box is for **Input**, and its purpose is to allow the user to **add a new candidate** to the voting system.

- **Prompt:** It asks the user to "**Enter candidate name:**"
- **Input Field:** The name "**Niloy**" has been typed into the text field.
- **Actions:** The user can click "**OK**" to confirm the entry and add Niloy as a candidate, or "**Cancel**" to close the dialog without making changes.

It is part of the process triggered by the "**Add Candidate**" button visible in the main window behind the dialog.

4.



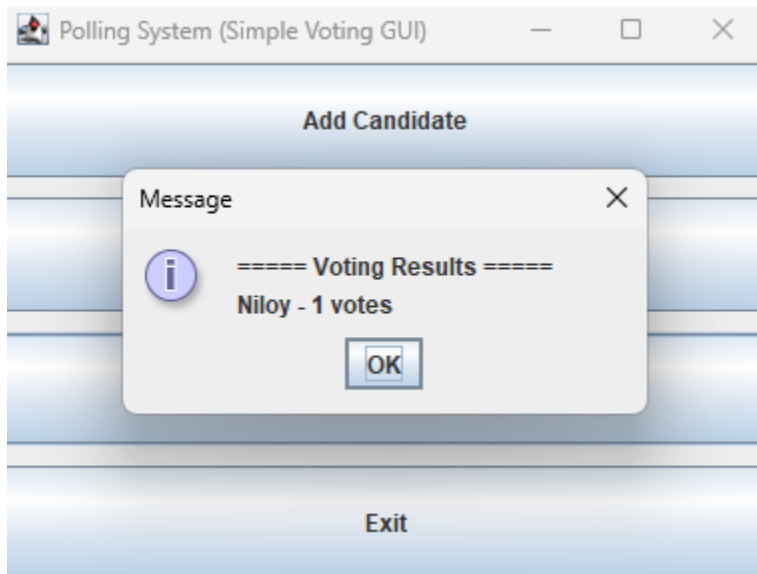
Explanation of the Image:

This image shows a "**Cast Vote**" dialog box from the same "**Polling System (Simple Voting GUI)**" application.

This dialog is used by the voter to submit their choice:

- **Title:** "**Cast Vote**" indicates the operation being performed.
- **Prompt:** It instructs the user to "**Select candidate to vote:**"
- **Input Field:** The user interface element is a **dropdown box** (indicated by the small downward arrow) which currently displays "**Niloy**" (the candidate who was added in the previous image's context).
- **Actions:** The user would select their desired candidate from the list, then click "**OK**" to record the vote, or "**Cancel**" to exit without voting.

5.



Explanation of the Image: Voting Results:

This image displays a "**Message**" dialog box from the "**Polling System (Simple Voting GUI)**", showing the **final outcome** of the voting process.

- **Title:** The dialog box is simply titled "**Message**", indicating it's an informational alert.
- **Content:** The main text, "**==== Voting Results =====**", clearly labels the purpose of the message.
- **Results:** The line "**Niloy - 1 votes**" shows the current count for the candidate named Niloy.
- **Action:** The user must click "**OK**" to acknowledge the results and dismiss the message.

This final screen is the culmination of the previous steps (adding Niloy as a candidate and casting one vote for him) and demonstrates the system's ability to count and display the results.

Github Link: <https://github.com/niloydas25010129/Java-Project.git>